ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ

Стекинг и блендинг в машинном обучении

Продвинутые методы ансамблирования моделей для максимального качества прогнозирования

Работу выполнили: студенты группы 5030102/30201

- Никоноров Артем
- Юров Иван

Зачем нужен стекинг? Какая задача решается?

Преодоление ограничений отдельных моделей и достижение синергетического эффекта.



Снижение дисперсии и смещения

Уменьшение генерализационной ошибки за счет агрегации некоррелированных предсказаний.



Дополнение моделей

Комбинирование линейных моделей, деревьев и нейросетей дает кумулятивный эффект.

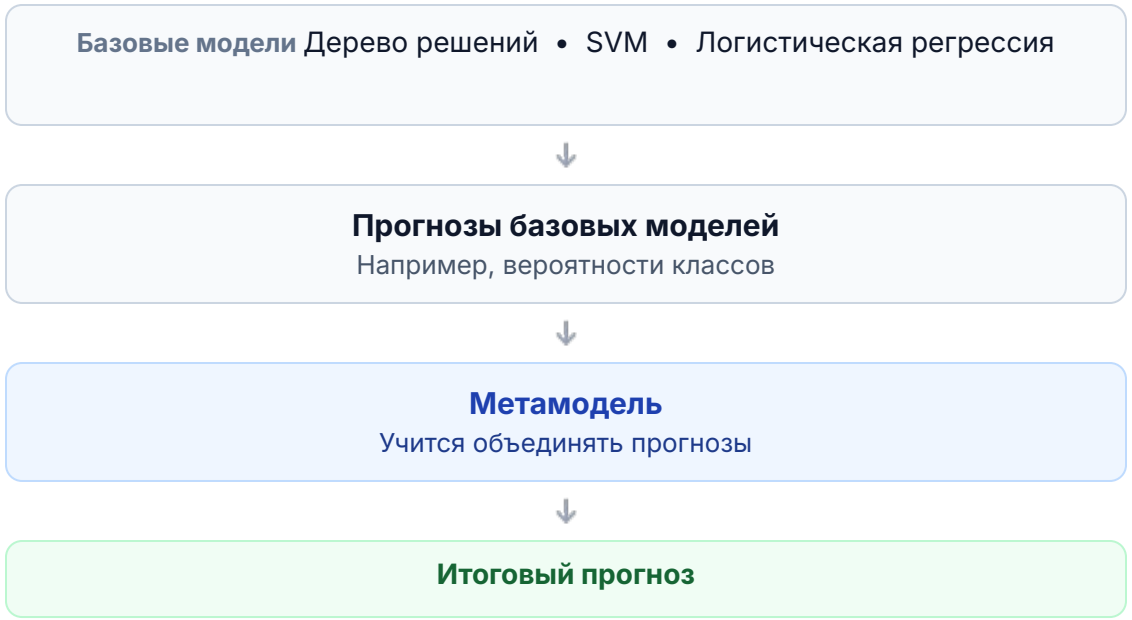


Улучшение качества метрик

Возможное улучшение выбранной метрики за счёт объединения прогнозов.

Стекинг и блендинг: общая идея

Принцип многоуровневого прогнозирования в ансамблях.



Стекинг (Stacking) — метамодель обучается на вневыборочных прогнозах базовых моделей, обычно сформированных с помощью кросс-валидации.

Блендинг (Blending) — метамодель обучается на прогнозах базовых моделей для отдельной валидационной выборки.

Схема работы стекинга

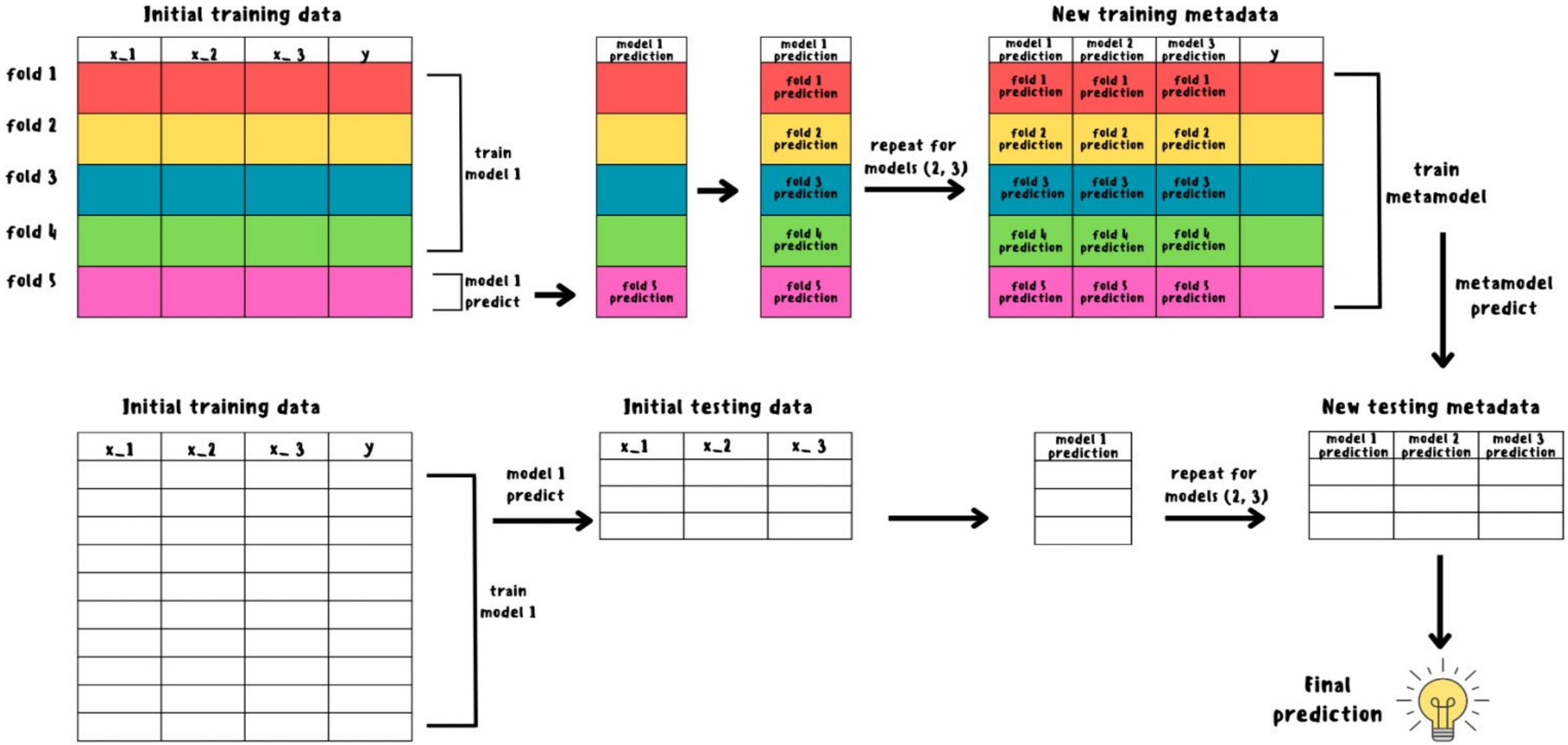
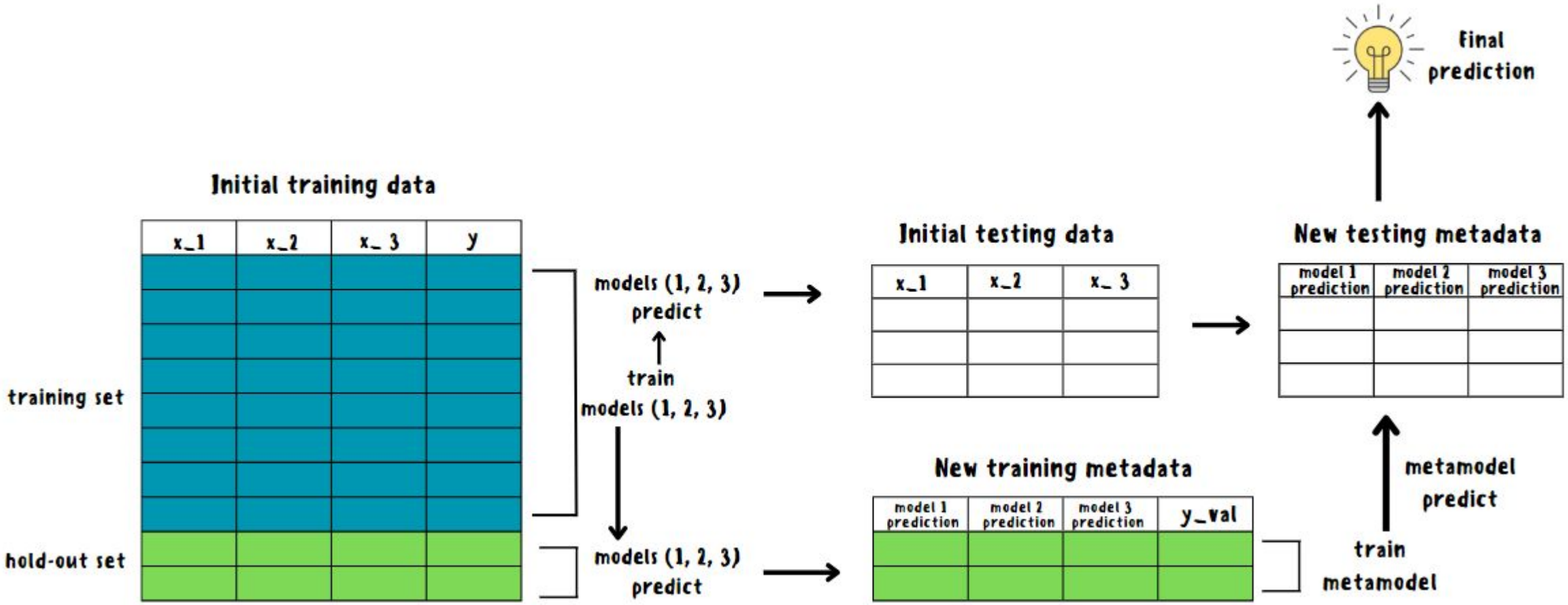


Схема работы блендинга



Архитектура стекинга: Уровни 0 и 1

Многоуровневый подход к обучению мета-модели над прогнозами базовых алгоритмов.

УРОВЕНЬ 0

Базовые модели

Random Forest, Градиентный бустинг, SVM и Neural Networks делают первичные независимые предсказания.

ПРОМЕЖУТОК

Матрица признаков

Ответы моделей уровня 0 формируют новый датасет (мета-признаки) для обучения верхнего уровня.

УРОВЕНЬ 1

Мета-модель

Простая линейная регрессия или Ridge находит оптимальные веса для комбинации прогнозов.

```python
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

# Определение базовых моделей (Уровень 0)
estimators = [
    ('dt', DecisionTreeClassifier(max_depth=4)),
    ('svc', SVC(probability=True, kernel='rbf'))
]

# Создание стекинга с метамоделью LogisticRegression (Уровень 1)
clf = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)

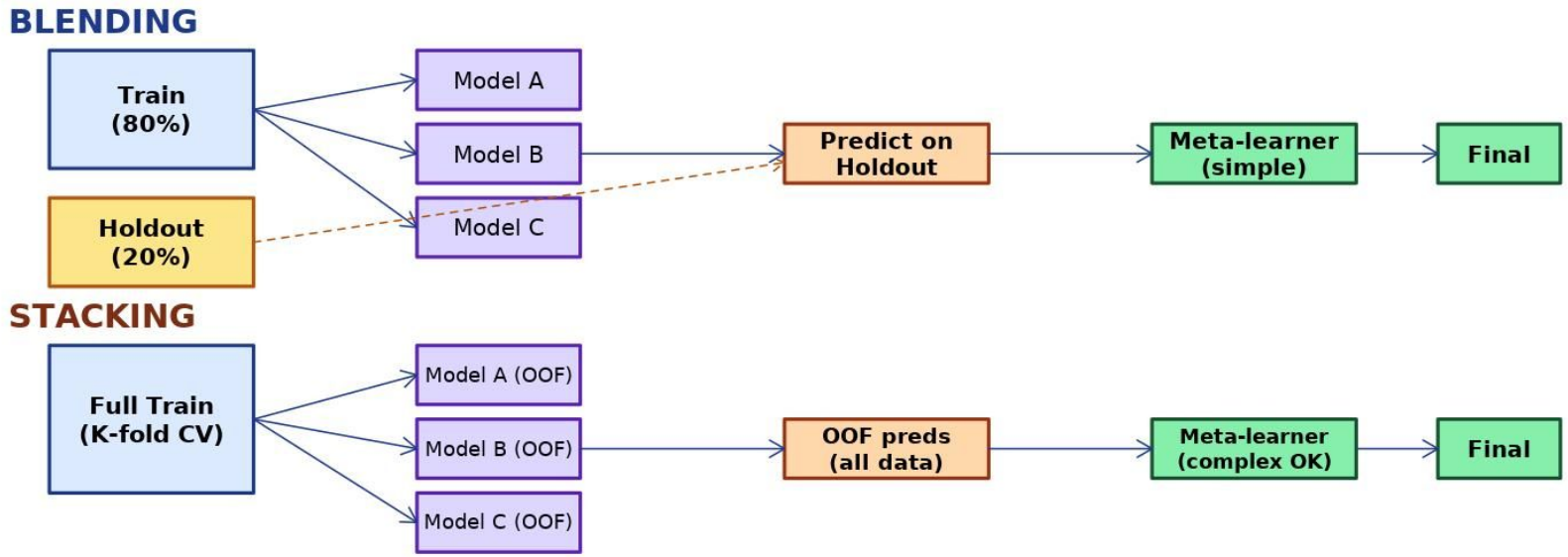
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

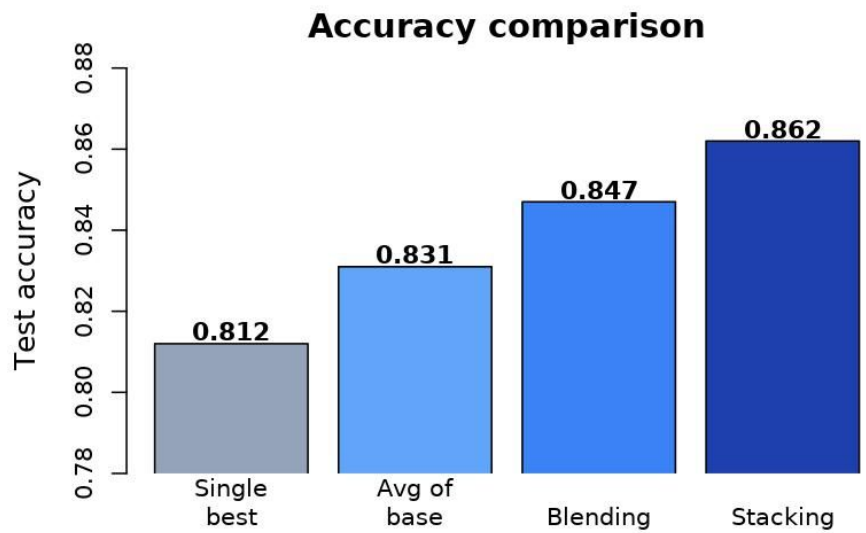
Стекинг против Блендинга

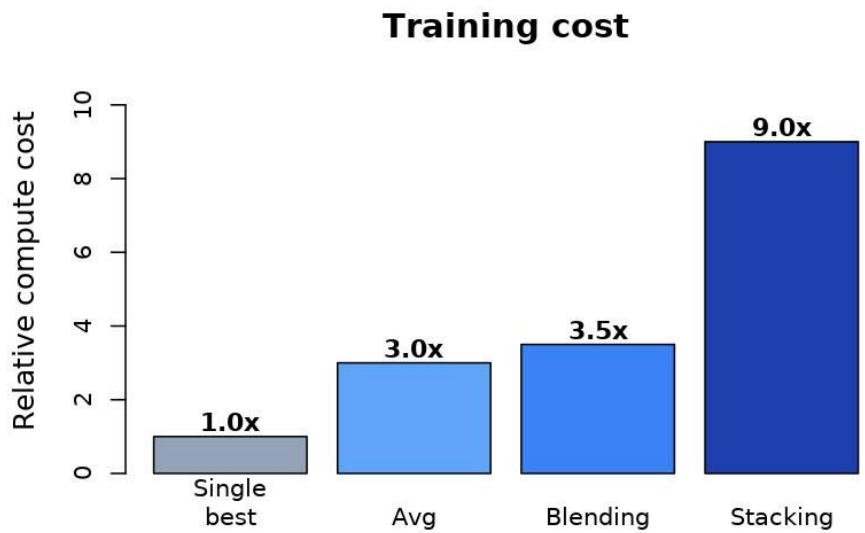
В чем ключевые отличия между двумя подходами ансамблирования?

Критерий сравнения	Стекинг (Stacking)	Блендинг (Blending)
Валидация данных	Использует кросс-валидацию (Out-of-Fold предсказания).	Использует разделение на Train и Validation (Hold-out).
Утечка данных	OOF-прогнозы помогают предотвратить утечку данных при обучении метамодели.	Часть обучающих данных резервируется для формирования прогнозов и обучения метамодели.
Вычислительная сложность	Выше из-за циклов кросс-валидации.	Ниже и быстрее в расчете (нет циклов CV).
Использование в практике	Стандарт для сложных соревнований и продакшна.	Удобен на очень больших датасетах для экономии времени.

Blending vs Stacking Architectures







Реализация на Python (Стекинг)

```python
import time
from sklearn.metrics import PredictionErrorDisplay
from sklearn.model_selection import cross_val_predict, cross_validate

fig, axs = plt.subplots(2, 2, figsize=(9, 7))
axs = np.ravel(axs)

for ax, (name, est) in zip(
    axs, estimators + [("Stacking Regressor", stacking_regressor)]
):
    scorers = {r"$R^2$": "r2", "MAE": "neg_mean_absolute_error"}

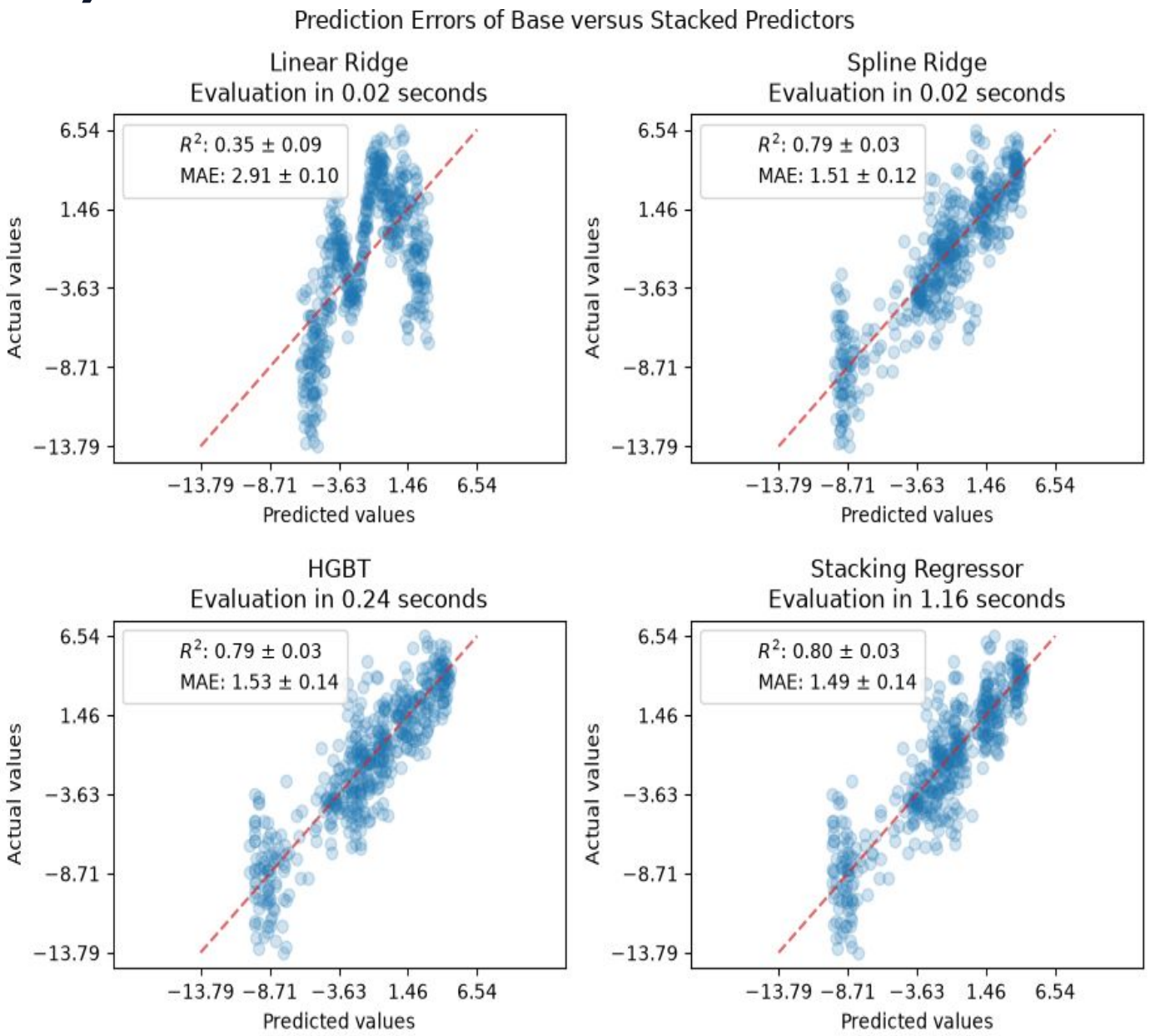
    start_time = time.time()
    scores = cross_validate(est, X, y, scoring=list(scorers.values()), n_jobs=-1)
    elapsed_time = time.time() - start_time

    y_pred = cross_val_predict(est, X, y, n_jobs=-1)
    scores = {
        key: (
            f"{np.abs(np.mean(scores[f'test_{value}'])):.2f}"
            r" $\pm$ "
            f"{np.std(scores[f'test_{value}']):.2f}"
        )
        for key, value in scorers.items()
    }

    display = PredictionErrorDisplay.from_predictions(
        y_true=y,
        y_pred=y_pred,
        kind="actual_vs_predicted",
        ax=ax,
        scatter_kwargs={"alpha": 0.2, "color": "tab:blue"},
        line_kwargs={"color": "tab:red"},
    )
    ax.set_title(f"{name}\nEvaluation in {elapsed_time:.2f} seconds")

    for name, score in scores.items():
        ax.plot([], [], " ", label=f"{name}: {score}")
    ax.legend(loc="upper left")

plt.suptitle("Prediction Errors of Base versus Stacked Predictors", y=1)
plt.tight_layout()
plt.subplots_adjust(top=0.9)
plt.show()
```



Реализация на Python (Блендинг)

```python
import numpy as np

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
import matplotlib.pyplot as plt


# Создаём искусственную задачу классификации
X, y = make_classification(
    n_samples=10_000,
    n_features=20,
    n_informative=15,
    n_redundant=5,
    random_state=7
)

# Сначала отделяем тестовую выборку
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y
)

# Разделяем оставшиеся данные на Train и Validation
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val,
    test_size=0.25, random_state=1, stratify=y_train_val
)

# Базовые модели
models = [
    DecisionTreeClassifier(max_depth=4, random_state=1),
    LogisticRegression(max_iter=3000),
    SVC(probability=True, random_state=1)
]
```

```python
models = [
    DecisionTreeClassifier(max_depth=4, random_state=1),
    LogisticRegression(max_iter=3000),
    SVC(probability=True, random_state=1)
]
for model in models:
    model.fit(X_train, y_train)

# Объединение вероятности базовых моделей в матрицы признаков
meta_val = np.hstack([
    model.predict_proba(X_val) for model in models
])

meta_test = np.hstack([
    model.predict_proba(X_test) for model in models
])

# Метамодель обучается на прогнозах Validation
meta_model = LogisticRegression(max_iter=3000)
meta_model.fit(meta_val, y_val)

y_pred = meta_model.predict(meta_test)

# Сравнение базовых моделей и блендинга
names = [
    "Decision Tree",
    "Logistic Regression",
    "SVC"
]

scores = {
    name: accuracy_score(y_test, model.predict(X_test))
    for name, model in zip(names, models)
}

scores["Blending"] = accuracy_score(y_test, y_pred)

for name, score in scores.items():
    print(f"{name}: {score:.2%}")
```

Вывод в консоль:

```
Decision Tree: 82.50%
Logistic Regression: 89.00%
SVC: 98.55%
Blending: 98.50%
```

Численный пошаговый пример (Блендинг)

Расчет взвешенной суммы мета-моделью для трех тестовых объектов.
Для простоты считаем, что веса уже подобраны на валидационной выборке. Применим полученную комбинацию к трём новым объектам.

Объект данных	Модель 1 (Дерево, $w_1 = 0.4$)	Модель 2 (Логист. регрессия, $w_2 = 0.6$)	Расчет мета-модели (Взвешенная сумма)	Итоговый прогноз
Объект №1	0.8 (Класс 1)	0.6 (Класс 1)	0.4 × 0.8 + 0.6 × 0.6 = 0.32 + 0.36 = **0.68**	**Класс 1 (> 0.5)**
Объект №2	0.2 (Класс 0)	0.4 (Класс 0)	0.4 × 0.2 + 0.6 × 0.4 = 0.08 + 0.24 = **0.32**	**Класс 0 (≤ 0.5)**
Объект №3	0.9 (Класс 1)	0.95 (Класс 1)	0.4 × 0.9 + 0.6 × 0.95 = 0.36 + 0.57 = **0.93**	**Класс 1 (> 0.5)**

ЧИСЛЕННЫЙ ПРИМЕР • ЗАДАЧА РЕГРЕССИИ

Стекинг: Исходные данные и Out-of-Fold

Условие задачи

Предсказание числового значения *y*.

Базовые модели: Модель 1 (Линейная регрессия) и Модель 2 (Дерево решений).

Метамодель: Линейная регрессия.

Кросс-валидация (2 фолда)

Разделяем выборку на две части:

1. Обучаем модели на первой части и прогнозируем вторую.
2. Меняем части местами и повторяем процесс.

Каждый объект получает прогноз от модели, которая не обучалась на нём.

Объект (ID)	Истинное (y)	Модель 1 (Z_1)	Модель 2 (Z_2)
1	10.0	9.5	10.2
2	15.0	14.0	14.8
3	20.0	18.5	21.0
4	25.0	24.0	24.5

Z_1, Z_2 — OOF-прогнозы базовых моделей, используемые как метапризнаки

ЧИСЛЕННЫЙ ПРИМЕР • РЕГРЕССИЯ (ПРОДОЛЖЕНИЕ)

Стекинг: Обучение метамодели и прогноз

ШАГ 2

Обучение метамодели

Формула: $\hat{y} = w_1 \cdot Z_1 + w_2 \cdot Z_2 + b$

При обучении метамодель подбирает веса на основе OOF-прогнозов и истинных ответов y. Для примера используем условные коэффициенты уже обученной метамодели: $w_1 = 0.4$, $w_2 = 0.6$, $b = 0.1$

ШАГ 3

Прогноз для нового объекта

Если $Z_1 = 30.0$, а $Z_2 = 29.0$:

$$\hat{y} = 0.4 \cdot 30.0 + 0.6 \cdot 29.0 + 0.1 = \mathbf{29.5}$$

Вывод: Метамодель взвешивает ответы с учетом индивидуальных особенностей моделей.

Альтернативные методы ансамблирования

Сравнение стекинга с другими классическими подходами.

Бэггинг (Bagging)

Параллельное обучение однотипных моделей на случайных подвыборках (пример: Random Forest).

Бустинг (Boosting)

Последовательное обучение моделей, где каждая исправляет ошибки предыдущей (XGBoost, CatBoost).

Голосование (Voting)

Простое усреднение прогнозов или мажоритарное голосование без обучаемого мета-алгоритма.

Сравнение методов ансамблей

Сводная таблица характеристик различных стратегий.

Метод	Вычислительная сложность	Риск переобучения	Качество прогноза
Стекинг / Блендинг	Высокая	Средний	**Потенциально высокое**
Бустинг	Средняя	Высокий	**Очень высокое**
Бэггинг	Низкая	Низкий	Высокое
Голосование (Voting)	Минимальная	Минимальный	Среднее

Качественное сравнение: характеристики зависят от данных и реализации

Оценка подхода: Плюсы и минусы

Что следует учитывать при внедрении стекинга в проект?

Преимущества

• Наивысшая точность прогнозирования на сложных данных.

• Гибкость в выборе базовых моделей разных архитектур.

• Возможное повышение устойчивости за счёт объединения моделей

Недостатки

• Высокая вычислительная сложность и время обучения.

• Риск переобучения мета-модели (требуется регуляризация).

• Сложность интерпретации и объяснения итоговых результатов.

Практические рекомендации

Правила успешного применения стекинга на практике.

Простая мета-модель

Стоит начинайте с простой метамодели - линейной регрессии или Ridge. Это помогает ограничить риск переобучения верхнего уровня.

Синергия сильных моделей

Сначала добейтесь максимум от мощных бустингов, а затем объединяйте их в стек с принципиально иными алгоритмами.

Контроль утечки

Строго следите за кросс-валидацией базовых моделей, чтобы мета-модель не обучалась на «подсмотренных» данных.

Спасибо за внимание!

Источники:
scikit-learn: StackingClassifier
scikit-learn: Combine predictors using stacking
Machine-learning.ru: Методы ансамблирования обучающихся алгоритмов
yandex: Ансамбли в машинном обучении